

```

import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import classification_report, roc_auc_score
import matplotlib.pyplot as plt
import seaborn as sns

# -----
# 2. Exploratory Data Analysis
# -----

print(df.head())
print(df.info())
print("\nMissing values per column:\n", df.isnull().sum())
print("\nSummary statistics:\n", df.describe())

# Check target balance
print("\nTarget distribution:\n", df['profile_view'].value_counts(normalize=True))

# Correlation heatmap
plt.figure(figsize=(10, 6))
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation Heatmap")
plt.show()

sns.pairplot(df, hue="profile_view", diag_kind="kde")
plt.suptitle("EDA: Distributions by Profile View", y=1.02)
plt.show()

# Example preprocessing (flexible)
df = df.dropna(subset=["profile_view"])
df = pd.get_dummies(df, drop_first=True)

# -----
# 3. Train/Test split
# -----
X = df.drop(columns="profile_view")
y = df["profile_view"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# -----
# 4. Logistic Regression
# -----
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

logit = LogisticRegression(max_iter=1000)
logit.fit(X_train_scaled, y_train)

y_pred_logit = logit.predict(X_test_scaled)
y_prob_logit = logit.predict_proba(X_test_scaled)[: , 1]

print("\n=== Logistic Regression ===")

```

```

print(classification_report(y_test, y_pred_logit))
print("AUC:", roc_auc_score(y_test, y_prob_logit))

# Coefficients interpretation
coef_df = pd.DataFrame({
    "Variable": X.columns,
    "Coefficient": logit.coef_[0]
}).sort_values(by="Coefficient", ascending=False)
print("\nLogistic Regression Coefficients:\n", coef_df)

# -----
# 5. Decision Tree
# -----
tree = DecisionTreeClassifier(max_depth=4, random_state=42)
tree.fit(X_train, y_train)

y_pred_tree = tree.predict(X_test)
y_prob_tree = tree.predict_proba(X_test)[:, 1]

print("\n=== Decision Tree ===")
print(classification_report(y_test, y_pred_tree))
print("AUC:", roc_auc_score(y_test, y_prob_tree))

# Plot the decision tree
plt.figure(figsize=(16, 8))
plot_tree(tree, feature_names=X.columns, class_names=["No View", "View"],
          filled=True, rounded=True, fontsize=9)
plt.show()

# -----
# 6. Feature importance (Decision Tree)
# -----
importances = pd.Series(tree.feature_importances_, index=X.columns)
importances.sort_values().plot(kind="barh", title="Decision Tree Feature Importances")
plt.show()

# -----
# 7. Compare results
# -----

results = pd.DataFrame({
    "Model": ["Logistic Regression", "Decision Tree"],
    "AUC": [roc_auc_score(y_test, y_prob_logit), roc_auc_score(y_test, y_prob_tree)],
    "Accuracy": [logit.score(X_test_scaled, y_test), tree.score(X_test, y_test)]
})
print("\n=== Model Comparison ===\n", results)

```